Лабораторная работа 4
ДИНАМИЧЕСКОЕ ПРОГРАММИРОВАНИЕ

ЦЕЛЬ РАБОТЫ: освоить общие принципы решения задач методом динамического программирования, сравнить полученные решения задач с рекурсивным методом.

ЗАДАНИЕ ДЛЯ ВЫПОЛНЕНИЯ:

Задание 1.
На языке C++ сгенерировать случайным образом строку букв латинского алфавита S1 длиной 300 символов и S2 длиной 200.

Задание 2.
Вычислить двумя способами (рекурсивно и с помощью динамического программирования) $L(prefix(S_1, k \times len(S_1)), prefix(S_2, k \times len(S_2))$ – дистанцию Левенштейна для $k = \left\{\frac{1}{25}, \frac{1}{20}, \frac{1}{15}, \frac{1}{10}, \frac{1}{5}, \frac{1}{2}, 1\right\}$, где $len(S)$ - длина строки S, $prefix(S,k)$ - строка состоящая из первых k символов строки S. (копии экрана и код вставить в отчет).

Задание 3.
Выполнить сравнительный анализ времени затраченного на вычисление дистанции Левенштейна для двух методов решения. Построить графики зависимости времени вычисления от k. (копии экрана и график вставить в отчет).

Задание 4.
Реализовать вручную пример вычисления дистанции Левенштейна при помощи рекурсивного алгоритма (в соответствии с вариантом) (каждый шаг алгоритма по примеру из лекции вставить в отчет).

Задание 5.
Нечетные варианты. Выполнить сравнительный анализ времени затраченного на решение задачи о наибольшей общей подпоследовательности для двух методов решения (рекурсивное решение, динамическое программирование). Две последовательности взять в соответствии с вариантом. Построить графики зависимости времени вычисления от k. **Отобразить ход решения в отчете** (по примеру из лекции) + код и копии экрана.

Четные варианты. Выполнить сравнительный анализ времени затраченного на решение задачи об оптимальной расстановке скобок при умножении нескольких матриц для двух методов решения (рекурсивное решение, динамическое программирование). Размерность матриц взять в соответствии с вариантом. Объяснить в отчете принцип расставления скобок по итоговой матрице + код + копии экрана.

Вариант	Задание 4		Задание 5	
1	град	город	ABCDEFG	BMDRFI
2	стол	стул	10*18, 18*23, 23*30, 30*28, 28*15, 15*40	
3	двор	дверь	AYCVWES	ECWMSP
4	двое	трое	10*25, 25*30, 30*55, 55*10, 10*30, 30*22	
5	чай	чаша	ARWXPUJ	WLPCAU
6	тень	день	10*22, 22*40, 40*33, 33*14, 14*20, 20*30	
7	звон	звук	AVTZNHI	AQTRYW
8	леса	луг	10*45, 45*60, 60*13, 13*30, 30*40, 40*11	
9	гора	вор	ABHCSUV	KIBOSV
10	река	рука	10*20, 20*30, 30*53, 53*20, 20*40, 40*50	
11	круг	кров	AVIEXAM	NIEHXM
12	таро	тара	10*11, 11*29, 29*42, 42*19, 19*30, 30*20	
13	лес	леска	ANPAFRE	ICBUFR
14	мать	дочь	10*35, 35*40, 40*23, 23*10, 10*30, 30*15	
15	груз	груша	AICPLKJ	PRBOLK
16	ноль	нож	10*50, 50*70, 70*33, 33*40, 40*50, 50*31	

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ

1. Вычисление дистанции Левенштейна

$$L(X,Y)=\begin{cases} len(Y),\ len(X)=0, \\ len(X),\ len(T)=0, \\ 0,\ len(X)=len(Y)=1 \wedge x=y, \\ 1,\ len(X)=len(Y)=1 \wedge x \neq y, \\ \min\begin{cases} L(cut(X),Y)+1, \\ L(X,cut(Y))+1, \\ \begin{cases} L(cut(X),cut(Y)),\ last(X)=last(Y), \\ L(cut(X),cut(Y))+1,\ last(X)\neq last(Y). \end{cases} \end{cases} \end{cases}$$

В предыдущем выражении используются символы len, cut и $last$. Разъясним их смысл:

$len(X)$ – количество символов в заданной строке. Например, $len(\text{"привет"}) = 6$;

$cut(X)$ – заданная строка без последнего символа. Например, $cut(\text{"привет"}) = \text{"приве"}$;

$last(X)$ – последний символ заданной строки. Например, $last(\text{"привет"}) = \text{"т"}$.

Поясним принцип применения этого рекуррентного соотношения на следующем примере.

Пусть необходимо вычислить $L(\text{"сор"}, \text{"спорт"})$. Тогда имеем следующую последовательность шагов:

1. $$L(\text{"сор"}, \text{"спорт"}) = \min\begin{cases} L(\text{"со"}, \text{"спорт"}) + 1, \\ L(\text{"сор"}, \text{"спор"}) + 1, \\ L(\text{"со"}, \text{"спор"}) + 1. \end{cases}$$

2. $$L(\text{"со"}, \text{"спорт"}) = \min\begin{cases} L(\text{"с"}, \text{"спорт"}) + 1, \\ L(\text{"со"}, \text{"спор"}) + 1, \\ L(\text{"с"}, \text{"спор"}) + 1. \end{cases}$$

3. $$L(\text{"сор"}, \text{"спор"}) = \min\begin{cases} L(\text{"со"}, \text{"спор"}) + 1, \\ L(\text{"сор"}, \text{"спо"}) + 1, \\ L(\text{"со"}, \text{"спо"}) + 1. \end{cases}$$

4. $$L(\text{"со"}, \text{"спор"}) = \min\begin{cases} L(\text{"с"}, \text{"спор"}) + 1, \\ L(\text{"со"}, \text{"спо"}) + 1, \\ L(\text{"с"}, \text{"спо"}) + 1. \end{cases}$$

5. $$L(\text{"с"}, \text{"спорт"}) = \min\begin{cases} L(\text{""}, \text{"спорт"}) + 1, \\ L(\text{"с"}, \text{"спор"}) + 1, \\ L(\text{""}, \text{"спор"}) + 1, \end{cases}$$

$L(\text{""}, \text{"спорт"}) = 5,\ L(\text{""}, \text{"спор"}) = 4.$

6. $$L(\text{"с"}, \text{"спор"}) = \min\begin{cases} L(\text{""}, \text{"спор"}) + 1, \\ L(\text{"с"}, \text{"спо"}) + 1, \\ L(\text{""}, \text{"спо"}) + 1, \end{cases}$$

$L(\text{""}, \text{"спор"}) = 4,\ L(\text{""}, \text{"спо"}) = 3.$

7. $$L("cop", "cno") = \min\begin{cases} L("co", "cno") + 1, \\ L("cop", "cn") + 1, \\ L("co", "cn") + 1. \end{cases}$$

8. $$L("co", "cno") = \min\begin{cases} L("c", "cno") + 1, \\ L("co", "cn") + 1, \\ L("c", "cn"). \end{cases}$$

9. $$L("cop", "cn") = \min\begin{cases} L("co", "cn") + 1, \\ L("cop", "c") + 1, \\ L("co", "c") + 1. \end{cases}$$

10. $$L("cop", "c") = \min\begin{cases} L("cop", "") + 1, \\ L("co", "c") + 1, \\ L("co", "") + 1, \end{cases}$$
$L("cop", "") = 3,\ L("co", "") = 2.$

11. $$L("c", "cno") = \min\begin{cases} L("", "cno") + 1, \\ L("c", "cn") + 1, \\ L("", "cn") + 1, \end{cases}$$
$L("", "cno") = 3,\ L("", "cn") = 2.$

12. $$L("co", "cn") = \min\begin{cases} L("c", "cn") + 1, \\ L("co", "c") + 1, \\ L("c", "c") + 1, \end{cases}$$
$L("c", "c") = 0.$

13. $$L("c", "cn") = \min\begin{cases} L("", "cn") + 1, \\ L("c", "c") + 1, \\ L("", "c") + 1, \end{cases}$$
$L("", "cn") = 2,\ L("c", "c") = 0,\ L("", "c") = 1.$

14. $$L("co", "c") = \min\begin{cases} L("c", "c") + 1, \\ L("co", "") + 1, \\ L("c", "") + 1, \end{cases}$$
$L("c", "c") = 0,\ L("co", "") = 2,\ L("c", "") = 1.$

15. $L("co", "c") = \min(1, 3, 2) = 1.$

16. $L("c", "cn") = \min(3, 1, 2) = 1.$

17. $L("co", "cn") = \min(2, 2, 1) = 1.$

18. $L("c", "cno") = \min(4, 2, 3) = 2.$

19. $L("cop", "c") = \min(2, 4, 3) = 2.$

20. $L("cop", "cn") = \min(2, 2, 2) = 2.$

21. $L("co", "cno") = \min(3, 2, 1) = 1.$

22. $L("cop", "cno") = \min(2, 3, 2) = 2.$

23. $L("c", "cnop") = \min(5, 3, 4) = 3.$

24. $L("c", "cnopm") = \min(6, 4, 5) = 4.$

25. $L("co", "cnop") = \min(4, 2, 3) = 2.$

26. $L("cop", "cnop") = \min(3, 3, 1) = 1.$

27. $L("co", "cnopm") = \min(5, 3, 4) = 3.$

28. $L("cop", "cnopm") = \min(4, 2, 3) = 2.$

```
// - Levenshtein.h
// -- дистанции   Левенштейна (динамическое программирование)
int levenshtein(
                  int lx,            // длина слова x
                  const char x[],    // слово длиной lx
                  int ly,            // длина слова y
                  const char y[]     // слово y
                );
// -- дистанции   Левенштейна (рекурсия)
int levenshtein_r(
                 int lx,            // длина строки x
                 const char x[],    // строка длиной lx
                 int ly,            // длина строки y
                 const char y[]     // строка y
                 );

// - Levenshtein.cpp
#include "stdafx.h"
#include <iomanip>
#include <algorithm>
#include "Levenshtein.h"
#define DD(i,j) d[(i)*(ly+1)+(j)]

int min3(int x1, int x2, int x3)
{ return std::min(std::min(x1,x2),x3); }

int levenshtein(int lx, const char x[],int ly, const char y[])
{
 int *d = new int[(lx+1)*(ly+1)];
 for(int i = 0; i  <= lx; i++) DD(i, 0) = i;
 for(int j = 0; j  <= ly; j++) DD(0, j) = j;
   for (int i = 1; i <= lx; i++)
       for (int j = 1; j <= ly; j++)
       {
         DD(i,j) = min3(DD(i-1, j) + 1, DD(i, j-1) + 1,
                        DD(i-1, j-1) + (x[i-1]==y[j-1]?0:1));
       }
 return DD(lx,ly);
}

int levenshtein_r(
```

```
                int lx, const char x[],
                int ly, const char y[]
                )
{
 int rc = 0;
 if        (lx == 0) rc = ly;
 else  if  (ly == 0) rc = lx;
 else  if  (lx == 1 && ly == 1 && x[0] == y[0]) rc = 0;
 else  if  (lx == 1 && ly == 1 && x[0] != y[0]) rc = 1;
 else  rc = min3(
             levenshtein_r(lx-1, x, ly,   y)+1,
             levenshtein_r(lx,   x, ly-1, y)+1,
             levenshtein_r(lx-1, x, ly-1, y)+(x[lx-1] == y[ly-1]?0:1)
               );
 return rc;
};


// --- main
//    вычисление дистанции (расстояния) Левенштейна
#include "stdafx.h"
#include <algorithm>
#include <iostream>
#include <ctime>
#include <iomanip>
#include "Levenshtein.h"
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 clock_t t1 = 0, t2 = 0, t3,t4;
 char x[] = "abcdefghklmnoxm",  y[] = "xyabcdefghomnkm";
 int  lx = sizeof(x)-1, ly = sizeof(y)-1;
 std::cout<<std::endl;
 std::cout<<std::endl<< "-- расстояние Левенштейна -----"<< std::endl;
 std::cout<<std::endl<< "--длина --- рекурсия -- дин.програм. ---"
     <<std::endl;
 for (int i = 8; i <  std::min(lx,ly); i++)
 {

  t1 = clock();levenshtein_r(i,x,i-2, y); t2 = clock();
  t3 = clock();levenshtein(i,x,i-2, y); t4 = clock();
  std::cout<<std::right<<std::setw(2)<<i-2<<"/"<<std::setw(2)<<i
       << "        "<<std::left<<std::setw(10)<<(t2-t1)
       <<"   "<<std::setw(10)<<(t4-t3)<<std::endl;
 }
 system("pause");
 return 0;
}
```

2. Решение задачи о расстановке скобок при перемножении матриц

```
// --- MultyMatrix.h
//    расстановка скобок
#pragma once
// расстановка скобок при умножении матриц
// функции возвращают минимальное количество операций умножения
#define OPTIMALM_PARM(x) ((int*)x)    // для представления 2мерного массива

int OptimalM(             // рекурсия
                  int i,          //  [in] номер первой матрицы
```

```
                    int j,          //  [in] номер последней матрицы
                    int n,          //  [in] количество матриц
                    const int c[],  //  [in] массив размерностей
                    int* s          //  [out] результат: позиции скобок
             );

int OptimalMD(                // динамическое программирование
                     int n,          //  [in] количество матриц
                     const int c[],  //  [in] массив размерностей
                     int* s          //  [out] результат: позиции скобок
              );


// --- MultiMatrix.cpp
//    расстановка скобок (рекурсия)
#include "stdafx.h"
#include <memory.h>
#include "MultiMatrix.h"
#define INFINITY  0x7fffffff
#define NINFINITY 0x80000000
int OptimalM(int i, int j, int n, const int c[], int *s)
{
 #define OPTIMALM_S(x1,x2)  (s[(x1-1)*n+x2-1])
 int o =INFINITY, bo = INFINITY;
 if (i<j)
 {
     for (int k = i; k<j;k++)
       {
        bo = OptimalM(i,k, n, c, s)+
             OptimalM(k+1,j,n, c, s)+ c[i- 1]*c[k]*c[j];
        if (bo < o)
            {
             o = bo;
             OPTIMALM_S(i,j) = k;
            }
       }
 }
 else  o = 0;
 return o;
 #undef OPTIMALM_S
};
// --- MultyMatrix.cpp (продолжение)
//    расстановка скобок (динамическое программирование)
int OptimalMD(int n, const int c[], int* s)
{
 #define OPTIMALM_S(x1,x2)  (s[(x1-1)*n+x2-1])
 #define OPTIMALM_M(x1,x2)  (M[(x1-1)*n+x2-1])
 int *M = new int[n*n], j = 0, q = 0;
 for (int i = 1; i <= n; i++) OPTIMALM_M(i,i) = 0;
 for (int l = 2; l <= n; l++)
 {
  for (int i = 1; i <= n-l+1; i++)
  {
   j = i+l-1;
   OPTIMALM_M(i,j) = INFINITY;
   for (int k = i; k <= j-1; k++)
   {
     q =  OPTIMALM_M(i,k) +  OPTIMALM_M(k+1,j)+c[i-1]*c[k]*c[j];
     if (q < OPTIMALM_M(i,j))
       {
          OPTIMALM_M(i,j) = q;  OPTIMALM_S(i,j)= k;
       }
   }
```

```
  }
 }
 return OPTIMALM_M(1,n);
 #undef OPTIMALM_M
 #undef OPTIMALM_S
};


// --- main
//    расстановка скобок
#include "stdafx.h"
#include <cmath>
#include <memory.h>
#include <iostream>
#include "MultiMatrix.h"   // умножение матриц

#define N 6
int main()
{

int Mc[N+1] = {5,10,15,20,25,30,35}, Ms[N][N], r = 0, rd = 0;

 memset(Ms,0,sizeof(int)*N*N);
 r = OptimalM(1, N, N, Mc, OPTIMALM_PARM(Ms));
 setlocale(LC_ALL, "rus");
 std::cout<<std::endl;
 std::cout<<std::endl<< "-- расстановка скобок (рекурсивное решение) "
           << std::endl;
 std::cout<<std::endl<< "размерности матриц: ";
   for (int i = 1; i <= N; i++) std::cout<<"("<<Mc[i-1]<<","<<Mc[i]<<") ";
 std::cout<<std::endl<< "минимальное количество операций умножения: " << r;
 std::cout<<std::endl<<std::endl<<"матрица S"<<std::endl;
   for (int i = 0; i <  N; i++)
         {
           std::cout<<std::endl;
           for (int j = 0; j <  N; j++)  std::cout<<Ms[i][j]<< "  "  ;
         }
 std::cout<<std::endl;

 memset(Ms,0,sizeof(int)*N*N);
 rd =  OptimalMD(N, Mc, OPTIMALM_PARM(Ms));
 std::cout<<std::endl
    << "-- расстановка скобок (динамичеое программирование) "<< std::endl;
 std::cout<<std::endl<< "размерности матриц: ";
       for (int i = 1; i <= N; i++)
          std::cout<<"("<<Mc[i-1]<<","<<Mc[i]<<") ";
 std::cout<<std::endl<< "минимальное количество операций умножения: "
           << rd;
 std::cout<<std::endl<<std::endl<<"матрица S"<<std::endl;
     for (int i = 0; i <  N; i++)
         {
           std::cout<<std::endl;
           for (int j = 0; j <  N; j++)  std::cout<<Ms[i][j]<< "  "  ;
         }
 std::cout<<std::endl<<std::endl;
 system("pause");

 return 0;
}
```

```
-- расстановка скобок (рекурсивное решение)

размерности матриц: (5,10) (10,15) (15,20) (20,25) (25,30) (30,35)
минимальное количество операций умножения: 13750

матрица S

0  1  2  3  4  5
0  0  2  3  4  5
0  0  0  3  4  5
0  0  0  0  4  5
0  0  0  0  0  5
0  0  0  0  0  0

-- расстановка скобок (динамичеое программирование)

размерности матриц: (5,10) (10,15) (15,20) (20,25) (25,30) (30,35)
минимальное количество операций умножения: 13750

матрица S

0  1  2  3  4  5
0  0  2  3  4  5
0  0  0  3  4  5
0  0  0  0  4  5
0  0  0  0  0  5
0  0  0  0  0  0

Для продолжения нажмите любую клавишу . . .
```

ОБЪЯСНИТЬ, КАК РАССТАВЛЯЮТСЯ СКОБКИ ПО ДАННЫМ МАТРИЦАМ

3. Решение задачи вычисления длины наибольшей общей подпоследовательности

```
// - LCS.h
// -- рекурсивное  вычисление длины LCS
int lcs (
              int lenx,          // длина   последовательности  X
              const char x[],    // последовательность X
              int leny,          // длина   последовательности  Y
              const char y[]     // последовательность Y
         );

// - LCS.cpp
// -- рекурсивное  вычисление длины LCS
#include "stdafx.h"
#include <algorithm>
#include "LCS.h"
int lcs (int lenx, const char x[],
         int leny, const char y[])
{
  int rc = 0;
  if (lenx > 0 && leny > 0)
  {
   if (x[lenx-1] == y[leny-1]) rc = 1 + lcs(lenx-1, x,leny-1, y);
   else rc = std::max(lcs(lenx, x,leny-1, y), lcs(lenx-1, x,leny, y));
  }
 return rc;         //длина LCS
}

// - main
// -- вычисления длины LCS
#include "stdafx.h"
#include <iostream>
```

```
#include "LCS.h"
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 char X[]="ALDC", Y[]="LADCM";
 std::cout<<std::endl<<"-- вычисление длины LCS для X и Y(рекурсия)";
 std::cout<<std::endl<<"-- последовательность X: "<< X;
 std::cout<<std::endl<<"-- последовательность Y: "<< Y;
 int s = lcs(
             sizeof(X)-1,  // длина   последовательности  X
             "ALDC",       // последовательность X
             sizeof(Y)-1,  // длина   последовательности  Y
             "LADCM"       // последовательность Y
            );
 std::cout<<std::endl<< "-- длина LCS: "<<s<<std::endl;
 system("pause");
 return 0;
}
```

Динамическое программирование

```
//- LCH.h
int lcsd(
            const char x[],  // последовательность X
            const char y[],  // последовательность  Y
char z[]         // наибольшая общая подпоследовательность
        );

//- LCS.cpp
#include "stdafx.h"
#include <cstring>
#include "LCS.h"
#define LCS_C(x1,x2)  (C[(x1)*(leny+1)+(x2)])
#define LCS_B(x1,x2)  (B[(x1)*(leny+1)+(x2)])
#define LCS_X(i)      (x[(i)-1])
#define LCS_Y(i)      (y[(i)-1])
#define LCS_Z(i)      (z[(i)-1])
enum Dart{TOP,LEFT,LEFTTOP};
void getLCScontent( int lenx, int leny, const char x[],
                    const Dart* B,
                    int n, int i, int j,   char z[])
{
 if ((i > 0 && j  > 0 && n > 0 ))
 {
   if (LCS_B(i,j) == LEFTTOP)
     {
        getLCScontent(lenx, leny,x, B, n-1, i-1, j-1, z);
        LCS_Z(n) = LCS_X(i);
        LCS_Z(n+1) = 0;
      }
   else if (LCS_B(i,j)== TOP)
          getLCScontent(lenx, leny,x, B, n, i-1, j, z);
   else getLCScontent(lenx, leny,x, B, n, i, j-1, z);
 }
};
int lcsd(const char x[], const char y[], char z[])
{
 int n;
 int lenx = strlen(x), leny = strlen(x),
     *C = new int[(lenx+1)*(leny+1)];
     Dart* B = new Dart[(lenx+1)*(leny+1)];
 memset(C,0,sizeof(int)*(lenx+1)*(leny+1));
 for (int i = 1; i <= lenx; i++)
```

```
  for(int j = 1; j <= leny; j++)
       if (LCS_X(i) == LCS_Y(j))
          {LCS_C(i,j) = LCS_C(i-1,j-1)+1;
           LCS_B(i,j) = LEFTTOP;}
       else if (LCS_C(i-1,j) >= LCS_C(i, j-1))
          {
           LCS_C(i,j) = LCS_C(i-1, j);
         LCS_B(i,j) = TOP;
          }
       else
          {
           LCS_C(i,j) = LCS_C(i, j-1);
           LCS_B(i,j) = LEFT;
          }
 getLCScontent(lenx, leny, x, B, LCS_C(lenx,leny), lenx, leny, z);
 return LCS_C(lenx,leny);
}
#undef LCS_Z
#undef LCS_C
#undef LCS_B
#undef LCS_X
#undef LCS_Y

// --- main
//    наибольшая общая подпоследовательность
#include "stdafx.h"
#include <iostream>
#include "LCS.h"

int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");

char z[100]="";
char x[] = "ABCBDAB",
     y[] = "BDCABA" ;

int l =  lcsd(x, y, z);
std::cout<<std::endl
    << "-- наибольшая общая подпоследовательость - LCS(динамическое"
    <<"программирование)"<< std::endl;
std::cout<<std::endl<<"последовательость X: " << x;
std::cout<<std::endl<<"последовательость Y: " << x;
std::cout<<std::endl<<"                LCS: " << z;
std::cout<<std::endl<<"          длина LCS: " << l;
std::cout<<std::endl;

 system("pause");
 return 0;
}
```

C		B	D	C	A	B	A
	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

B		B	D	C	A	B	A
A		↑	↑	↑	↖	←	↖
B		↖	←	←	↑	↖	←
C		↑	↑	↖	←	↑	↑
B		↖	↑	↑	↑	↖	←
D		↑	↖	↑	↑	↑	↑
A		↑	↑	↑	↖	↑	↖
B		↖	↑	↑	↑	↖	↑

ОБЪЯСНИТЬ, КАК ЗАПОЛНЯЮТСЯ МАТРИЦЫ И ЧТО ОНИ ОЗНАЧАЮТ

Вопросы для защиты:

1. В каких областях используется динамическое программирование?
2. В чем заключается задача динамического программирования?
3. Чем аддитивная функция отличается от мультипликативной?
4. Каков принцип оптимальности Беллмана?
5. Что такое рекурсивный алгоритм?
6. Что такое рекурсивная функция?
7. Что такое системный стек?
8. Объясните понятие «глубина рекурсии»
9. Поясните своими словами схему решения задачи по принципу «разделяй и властвуй»
10. Что такое редакционное расстояние?
11. Что такое подпоследовательность и как её можно получить из последовательности?